

Three ways to say “that is not normal”

mean $\pm k\sigma$, isolation forest and PCA reconstruction error — from the ground up

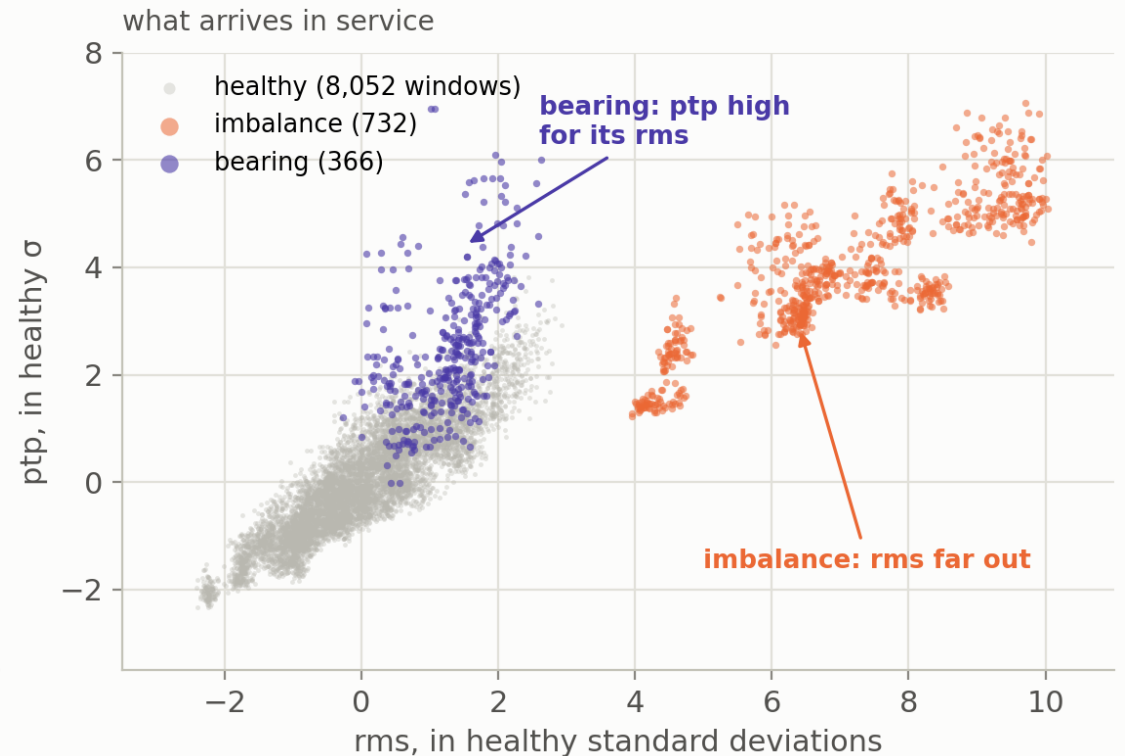
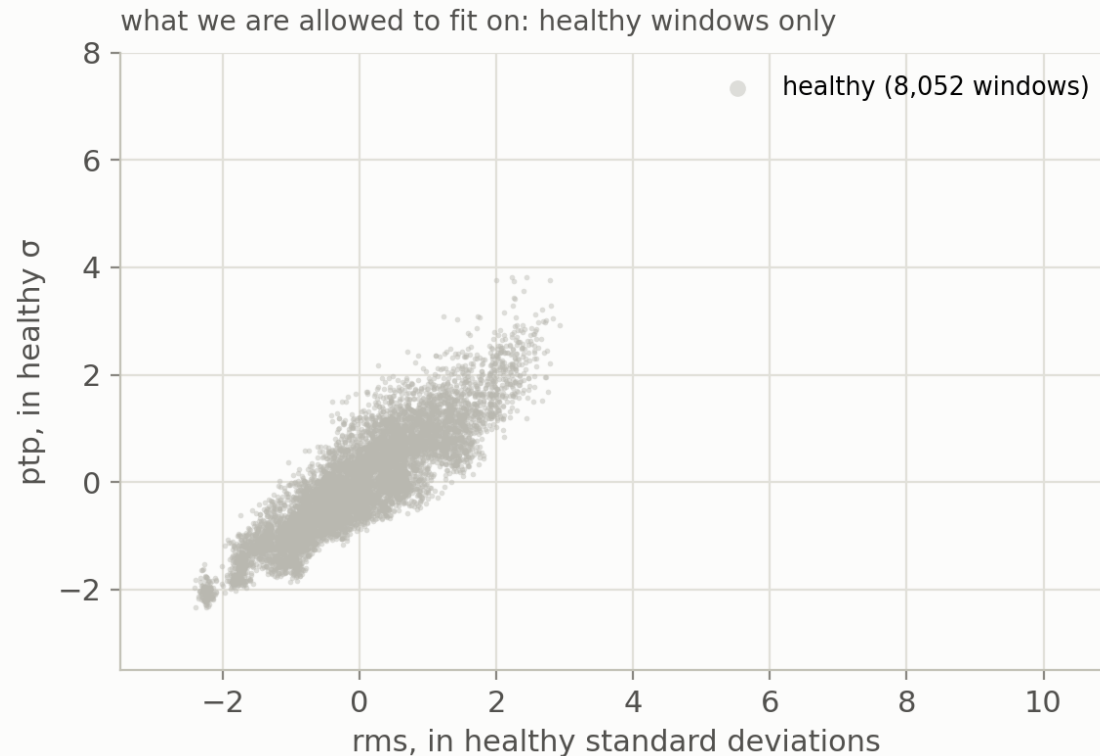
- What “normal” means for one feature — and why the threshold is never 3σ
- mean $\pm k\sigma$ draws a box; Mahalanobis draws an ellipse that leans with the data
- The isolation forest plays a game of random cuts — and hits a ceiling
- PCA keeps the directions healthy data uses, and measures what is left over
- All four on the rig: Lecture 10’s table reproduced, plus the two it did not have
- What happens when your “healthy” training data is not quite healthy

Notebook: `Lecture10_Supplement1_Anomaly_Detection.ipynb`

Supplementary material — background for Lecture 10, not new examinable content

The job: score, then threshold

- We can only learn what healthy looks like — then ask of every new window: how unlike healthy is this?

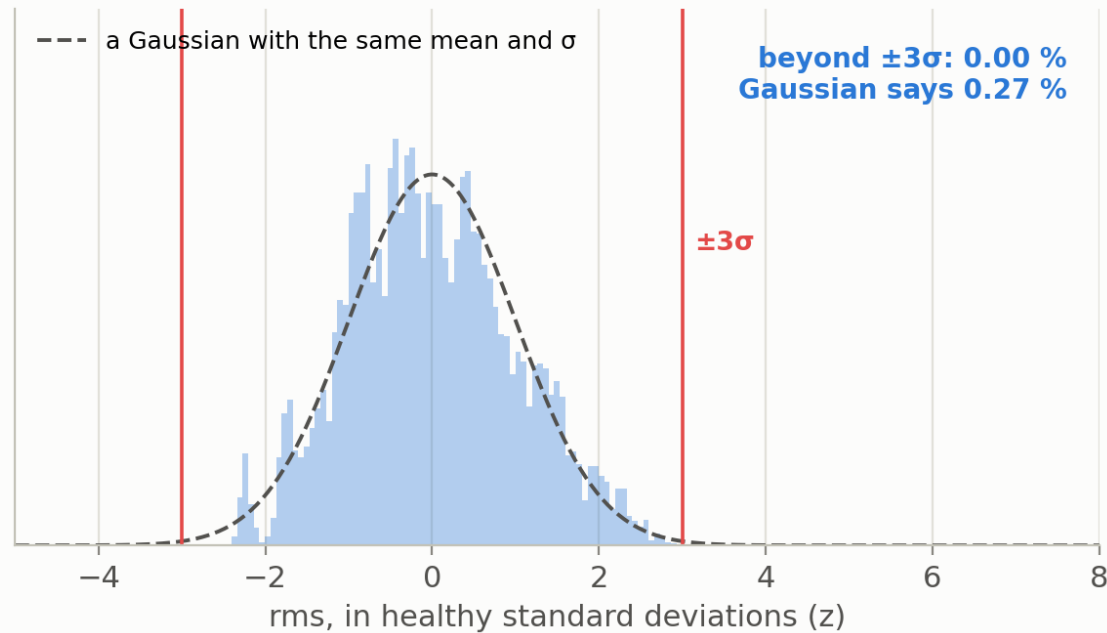


Every method has two steps. The method gives the **SCORE**. The **THRESHOLD** is yours — a quantile of healthy scores — and it decides your false-alarm rate.

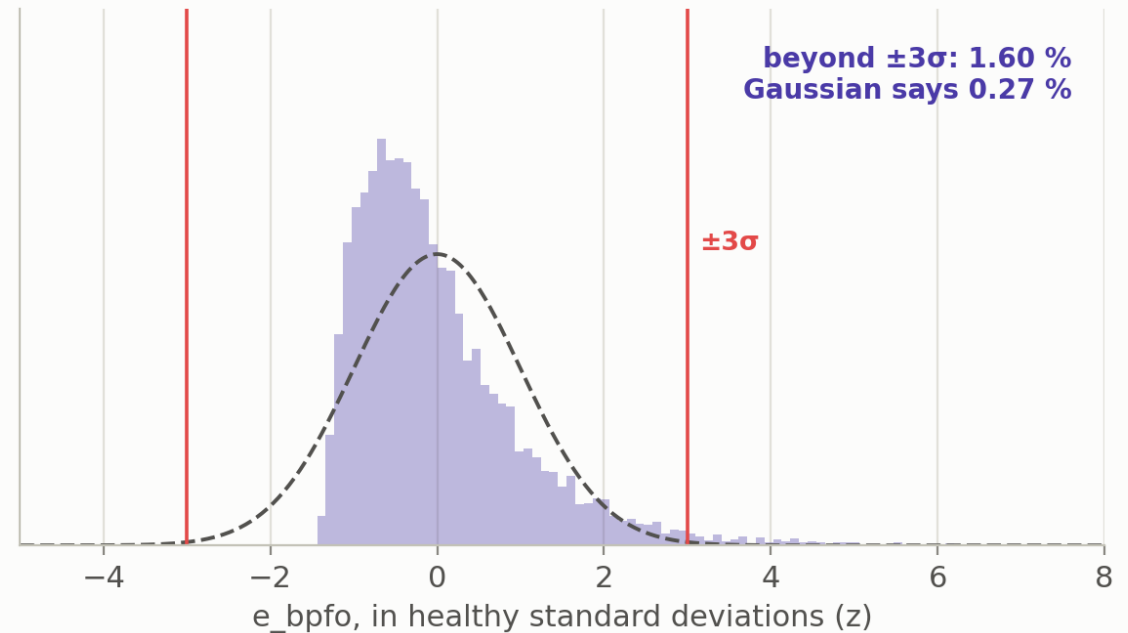
“Beyond 3σ ” is a promise only a Gaussian keeps

- $z = (x - \mu) / \sigma$. For a Gaussian, 0.27 % of values lie beyond $\pm 3\sigma$ — but our features are not Gaussian.

rms: symmetric, light tails



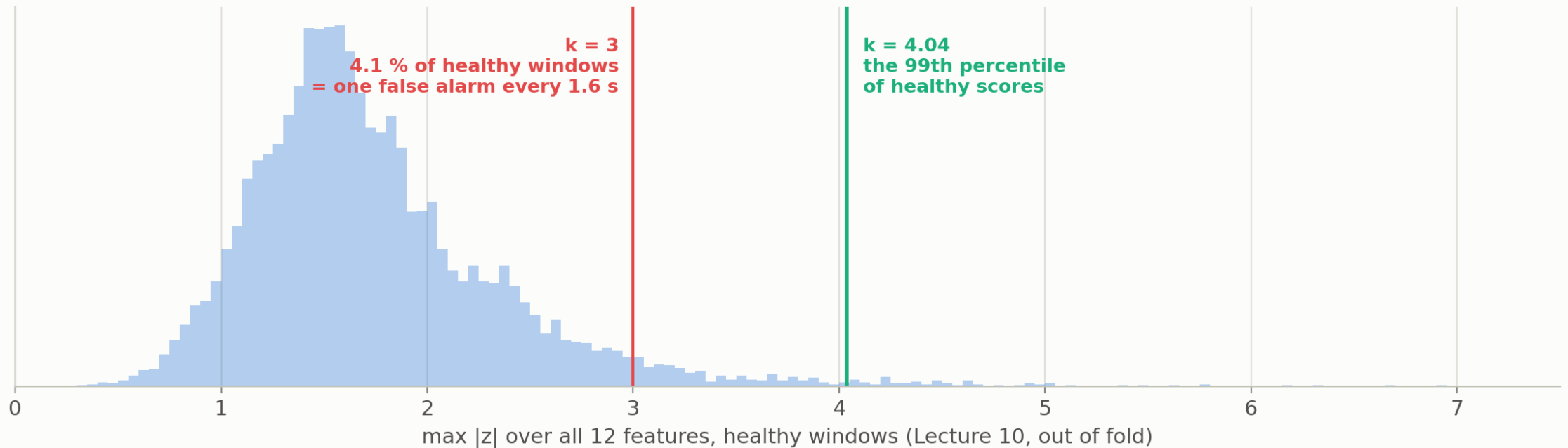
e_bpfo: skewed — the tail is long on one side



Same “ 3σ ”, false-alarm rates of 0.00 % and 1.60 %. So a threshold is never “ $k = 3$ ” — it is the value only 1 % of healthy windows exceed.

With twelve features, k is not 3

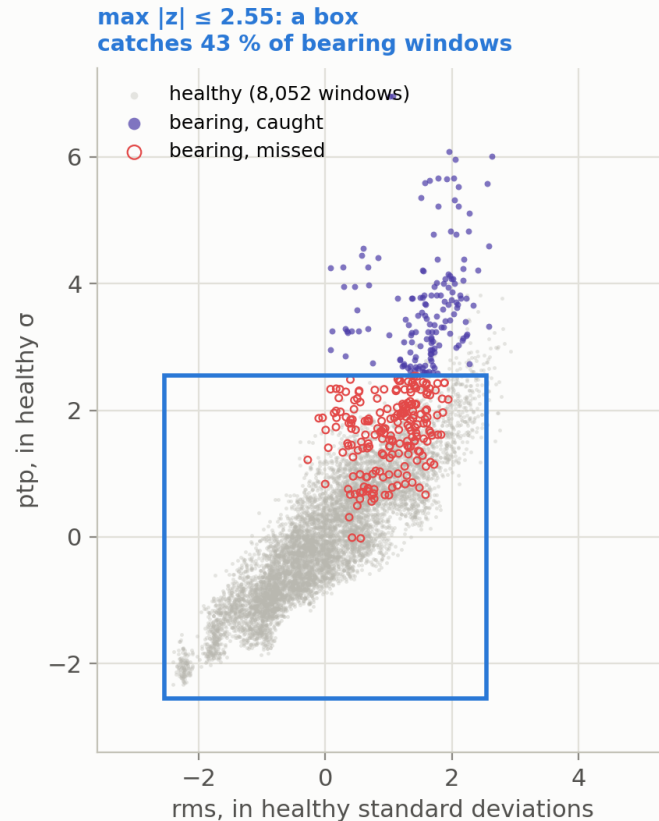
- Score = $\max |z|$ over twelve features: twelve chances to land beyond 3, and skewed features add more.



The data's own answer is $k = 4.04$ — exactly the threshold in Lecture 10's alarm spec. $k = 3$ would alarm once every 1.6 seconds on a healthy machine.

The box, and the ellipse that leans with the data

- $\max |z| \leq k$ is a square; the healthy cloud is a tilted cigar — and the bearing windows sit in its empty corner.



Same 1 % false alarms. The box catches 43 % of bearing windows; the ellipse catches 54 %. Nothing about the data changed — only the shape of “normal”.

Mahalanobis: the same question, asked of all features together

```
z      = (x - mean) / std      # the same z-scores as before
C      = correlation matrix of healthy z
score = sqrt( z · C-1 · z )    # distance, allowing for correlation
```

What it does

Divides each direction by how much healthy data varies along it. A direction the data spreads a lot along is cheap to move in; a direction it never moves in is expensive. That turns the box into an ellipse.

The trap you will hit on the rig

rms and std are the same number once the mean is removed (they differ by $1e-16$). So C has rank 11 of 12 and cannot be inverted. Use the pseudo-inverse, which ignores directions healthy data never moves in.

Not in Lecture 10 — and on this rig it beats all three

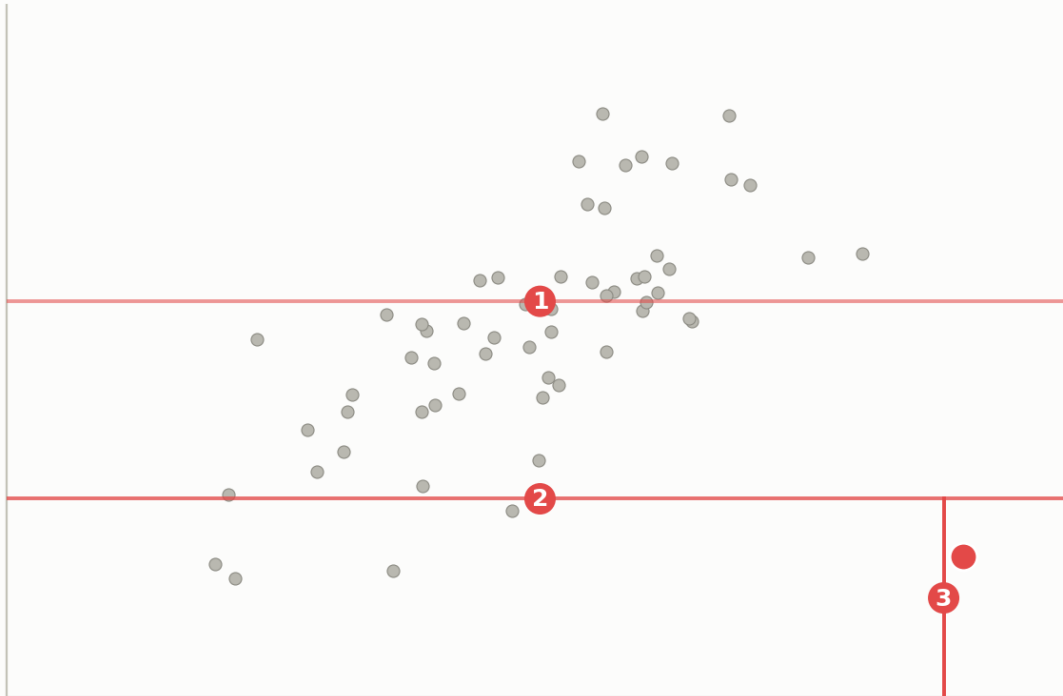
AUC 0.972, bearing 0.631, imbalance 1.000. The healthy data really is one correlated cloud, so the method that assumes exactly that wins.

Twelve features do not mean twelve independent directions. Mahalanobis needs you to know that; PCA makes it the whole method.

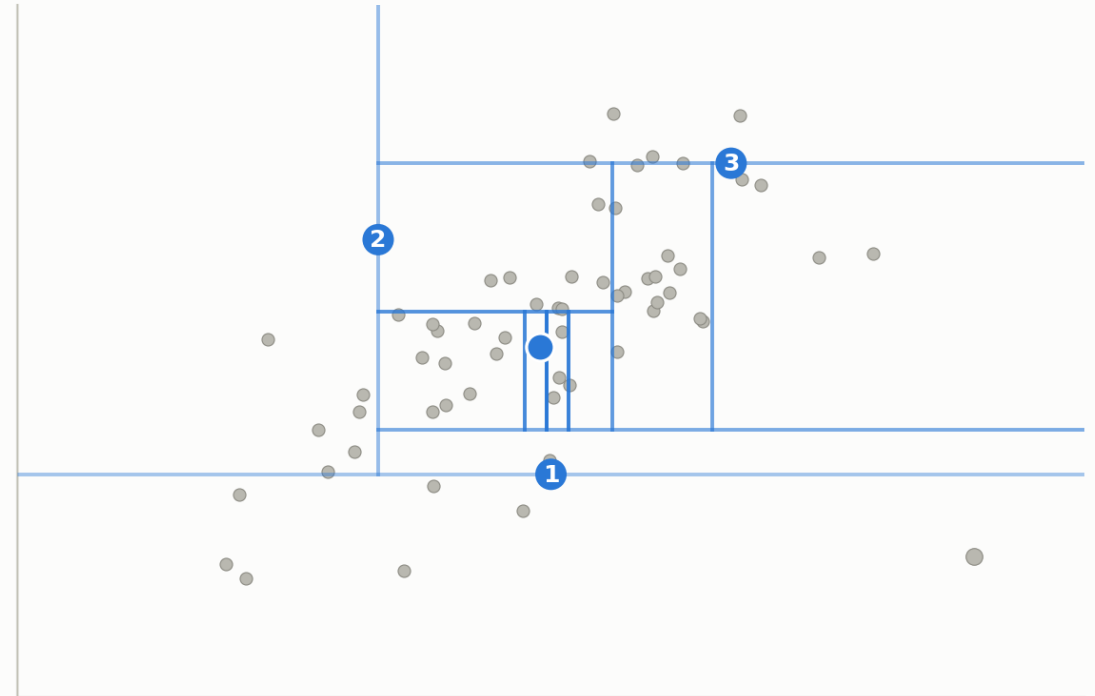
The isolation forest plays a game of random cuts

- Random feature, random cut, keep your window's side, repeat until it is alone — and count the cuts.

**the odd one: fenced off by 3 random cuts
(average over 400 random trees: 3.3)**



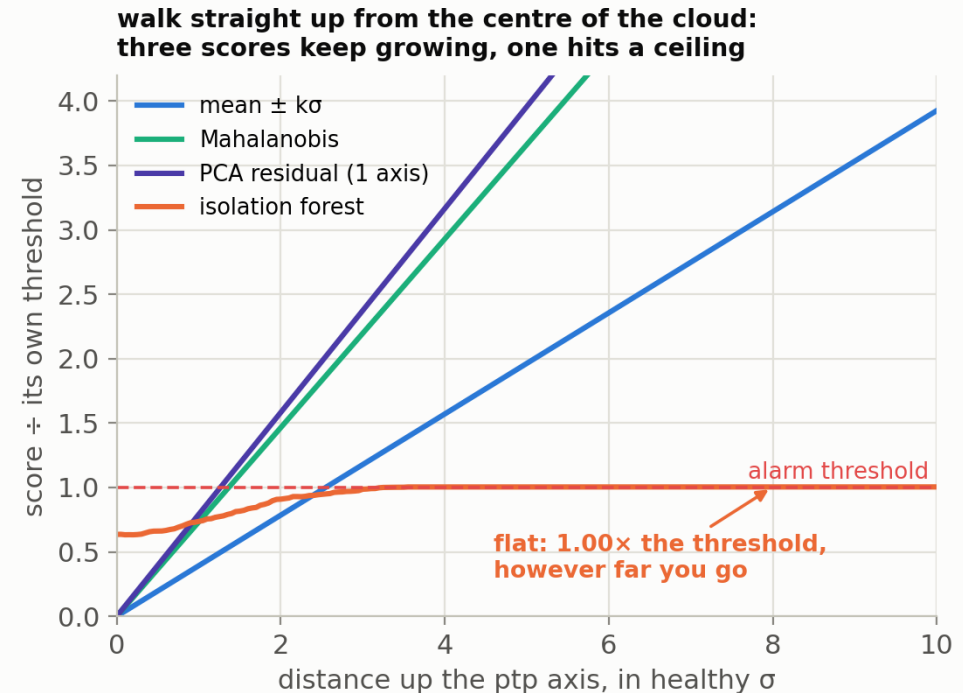
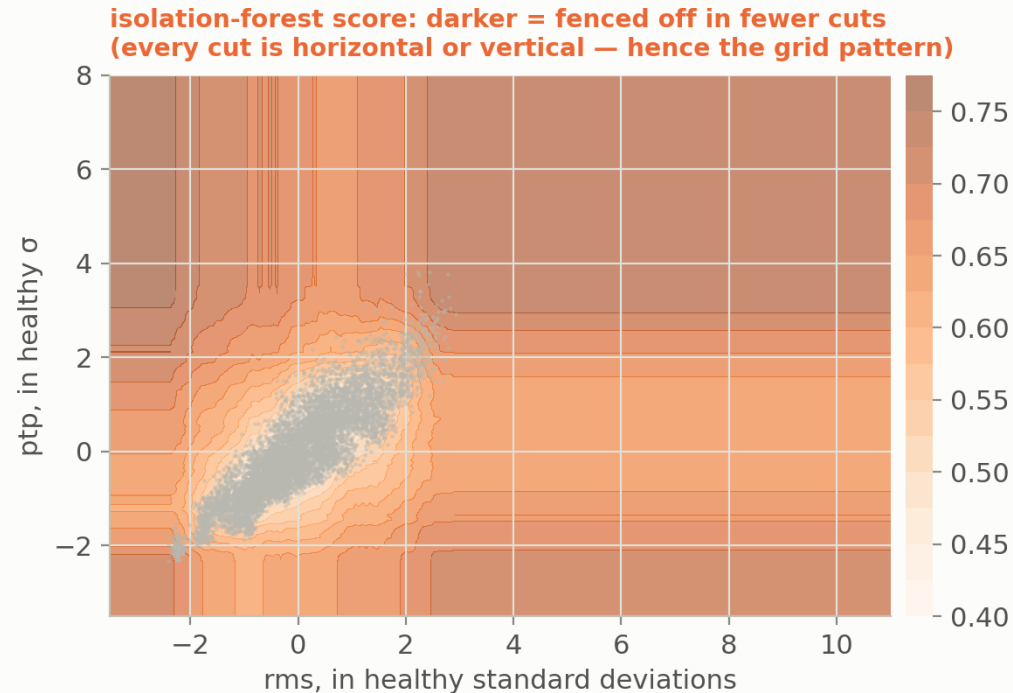
**an ordinary one: fenced off by 10 random cuts
(average over 400 random trees: 9.7)**



Averaged over 400 random trees: 3.3 cuts for the odd point, 9.7 for an ordinary one. That count, averaged over 200 trees, is the score. Nothing is assumed about shape.

The forest knows “outside”, not “how far outside”

- Outside all the healthy data, one cut isolates a window — the minimum. 3σ out and 30σ out score the same.



In two dimensions: no headroom

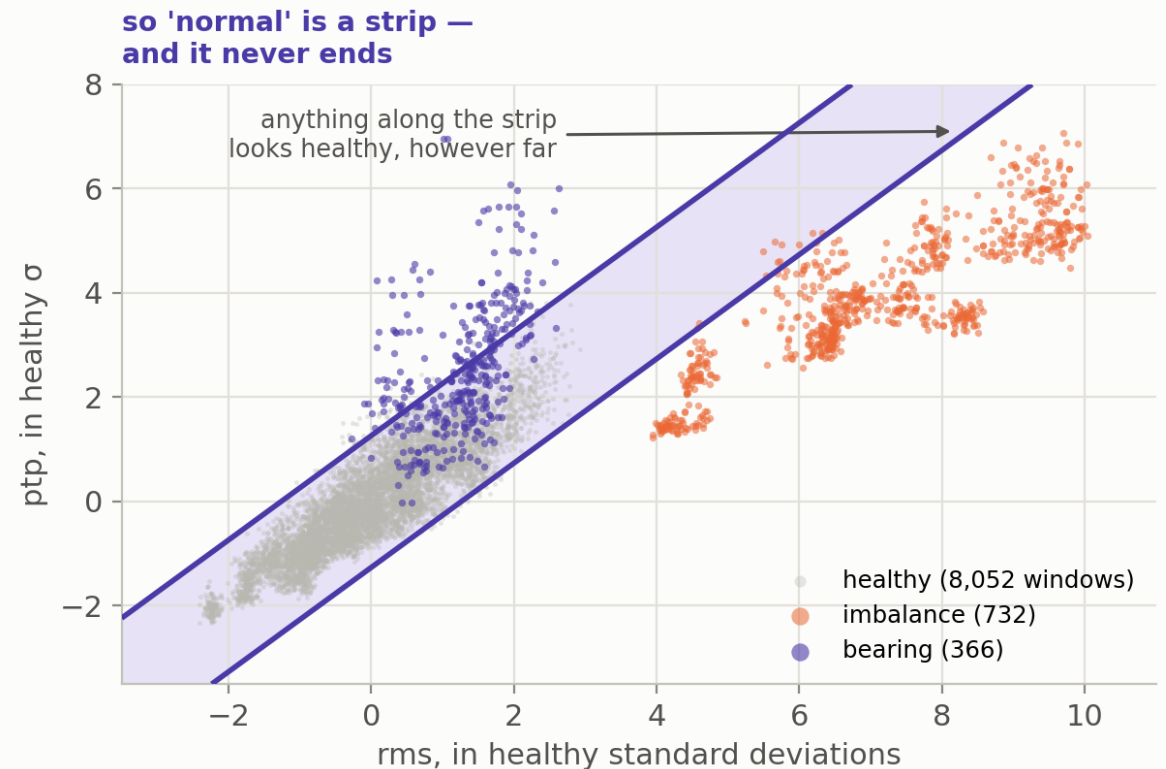
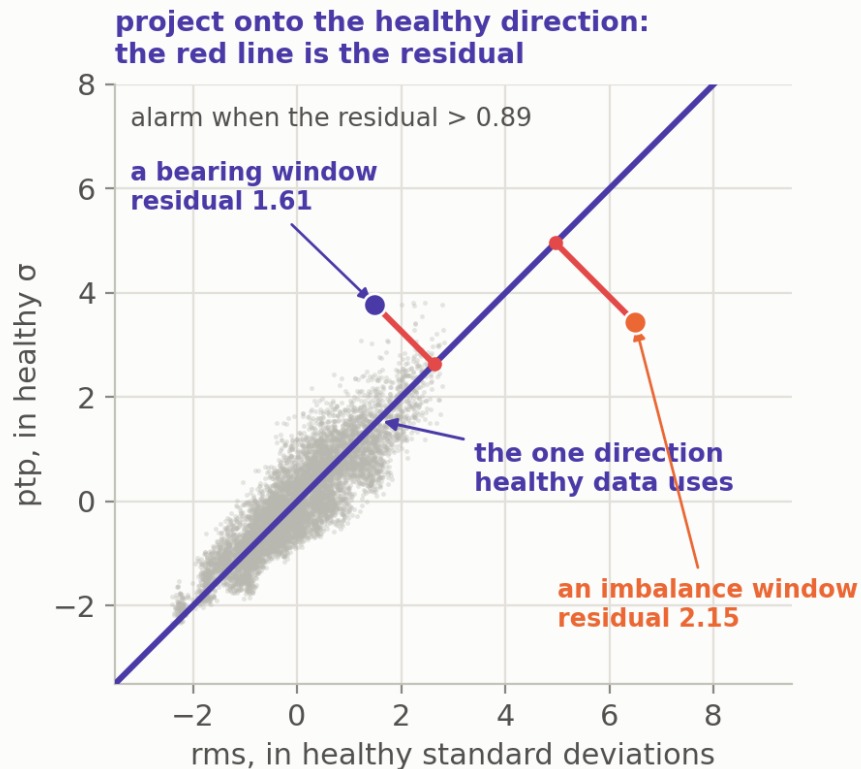
The healthy fringe already reaches the ceiling: threshold 0.677, ceiling 0.680. Only 25 % of bearing windows get above it.

In twelve: one-feature faults slip through

Push kurt 50σ out, leave eleven features ordinary: $\max |z|$ says 50; the forest says $0.78\times$ its threshold — healthy.

PCA: keep the directions healthy data uses

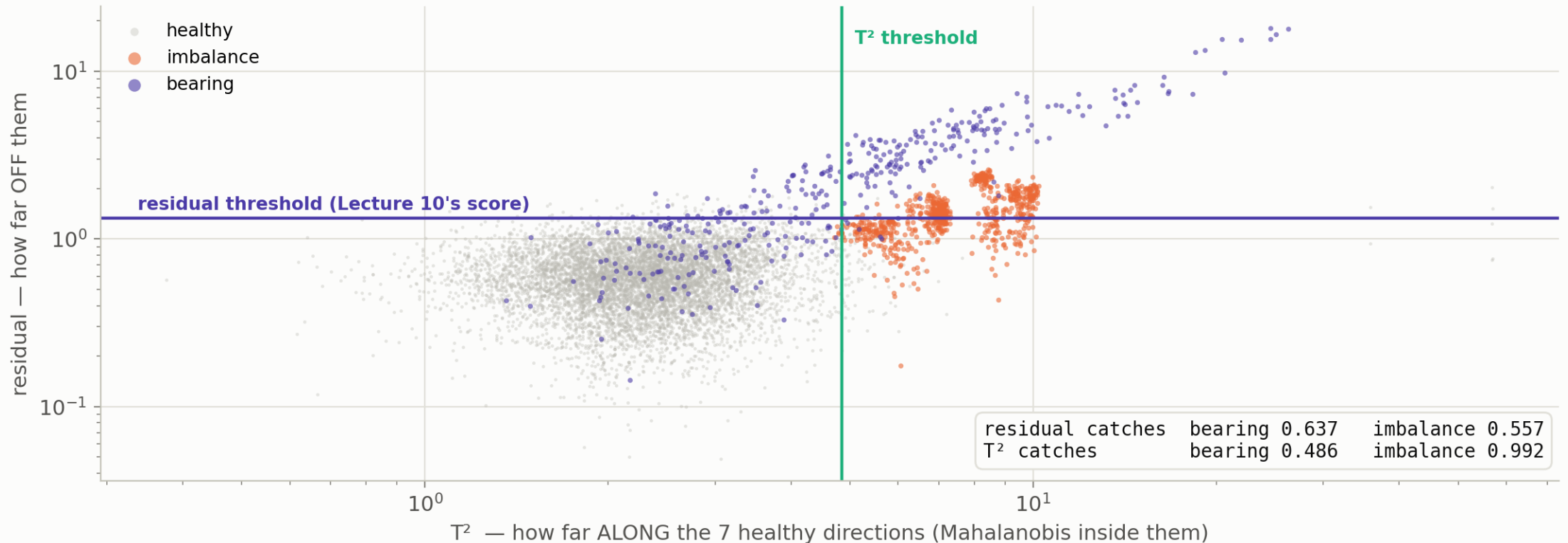
- Healthy rms and ptp move along one direction (94 % of the variance). The residual is how far a window lies off it.



The residual asks: how much of this window can healthy behaviour NOT explain? So “normal” is a strip — and it never ends.

PCA's blind spot — and the half Lecture 10 did not use

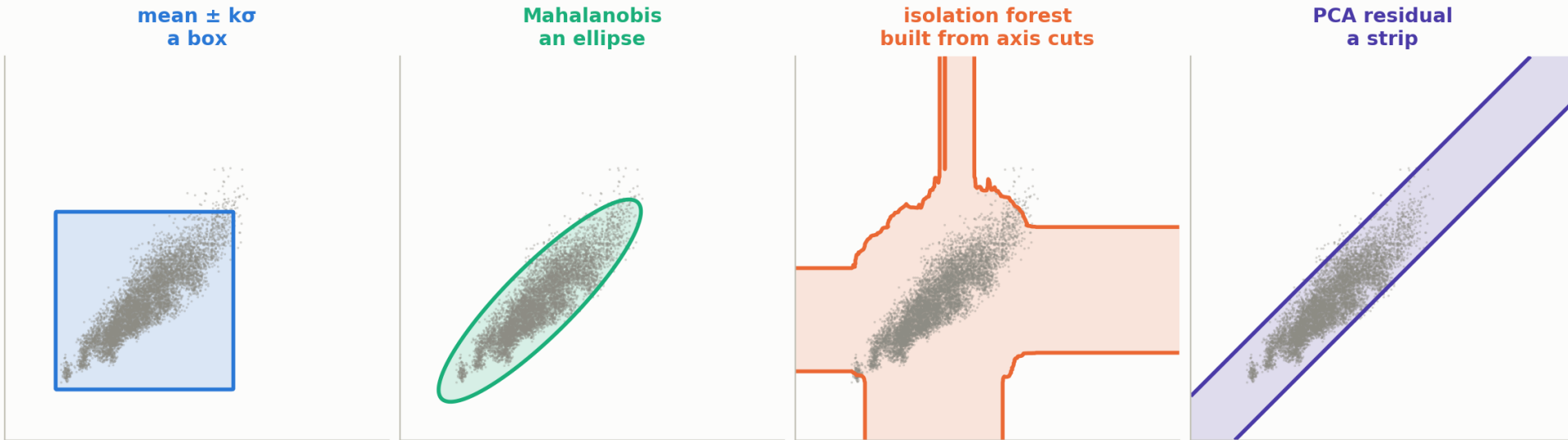
- On 12 features PCA keeps 7 directions. Imbalance moves ALONG them — it is just more of what healthy does.



The median imbalance window has a residual 1.04× its threshold — right on the line — but a T^2 of 1.46×. Industrial PCA monitoring watches both.

Four shapes of “normal”

- Same healthy data, each method at its 99th-percentile threshold. Know the shape, and you can predict the misses.



box

misses faults wrong only together

ellipse

misses data with several clusters

random cuts

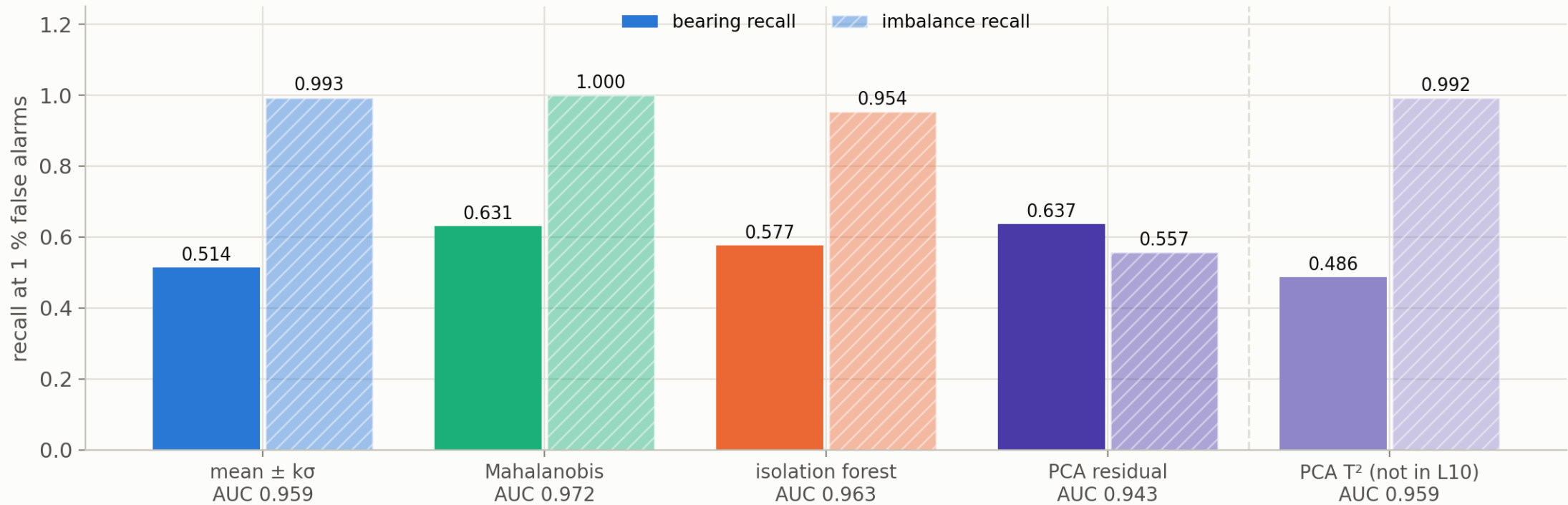
misses “how far”, and one-feature faults

strip

misses faults along healthy directions — unless you add T^2

On the rig: Lecture 10's table, plus the two it did not have

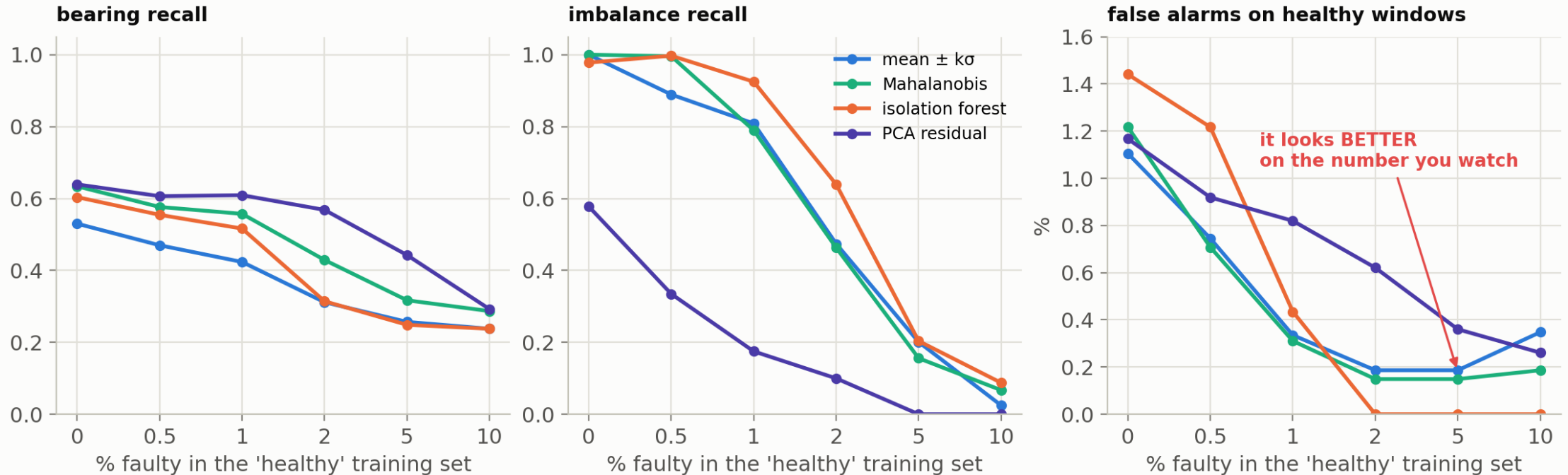
- Lecture 10's exact protocol and folds: columns 1, 3 and 4 match its slides to the third decimal.



Why Lecture 11 still deploys mean $\pm$ $k\sigma$: 25 numbers, it names the feature that tripped — which an ellipse cannot — and 0.514 vs 0.631 is inside the fold-to-fold spread Lecture 10 measured.

When “healthy” is not quite healthy

- Slip a few faulty windows into the training set, then fit and threshold exactly as if it were clean.



The threshold is the first casualty

A 99th percentile is set by the top 1 % of the data. Past 1 % contamination, it is set by the faults. For mean $\pm k\sigma$, k climbs 3.92 $\rightarrow$ 5.42 at 2 %.

And it looks better while it happens

At 2 %, mean $\pm k\sigma$ catches 0.311 / 0.474 (bearing / imbalance, was 0.530 / 1.000) — and false alarms FALL from 1.1 % to 0.2 %.

Which one?

mean $\pm k\sigma$

25 numbers — fits the ESP32

names the feature that tripped

misses faults that are wrong only together

Mahalanobis

168 numbers — fits the ESP32

best on this rig

gives one distance, no feature name; assumes one cloud

isolation forest

200 trees — cloud only

assumes no shape; copes with several clusters

saturates; weak on one-feature faults

PCA (+ T^2)

~110 numbers — fits the ESP32

residual and T^2 see opposite faults

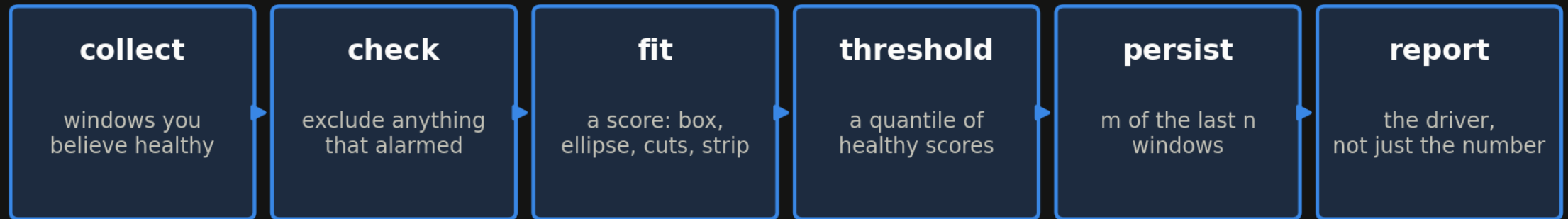
use both; the most fragile to contamination

A sensible default for a machine like this one

mean $\pm k\sigma$ on the device — cheap, local, and it tells the technician which feature. Mahalanobis, or PCA with both T^2 and the residual, in the cloud — where the extra numbers cost nothing and the correlations are worth catching.

Whichever you pick: fit on data you have checked, exclude anything that alarmed, and re-measure detection — not just false alarms — every time you refit.

Know the shape, and you know the misses



the choice of score is one box of six — the other five decide whether it works

1. Score, then threshold. The threshold is a quantile of healthy scores — never a textbook 3σ .
2. $\text{mean} \pm k\sigma$ is a box: it judges features one at a time.
3. Mahalanobis is an ellipse: it judges them together — and on correlated data it wins.
4. The isolation forest counts random cuts: no shape assumed, but it saturates.
5. PCA's residual is a strip that misses faults along healthy directions — add T^2 .
6. Contamination deafens every method, and false alarms improve while it happens.